Bancos de Dados Não-Relacionais

Aula 08: MongoDB Aggregation Framework – Pipelines Analíticos e Business Intelligence

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Ceará
Campus Tauá

10 de Novembro de 2026

Agenda do Encontro

Paradigma de Processamento em Pipeline (Dataflow)

Estágios Fundamentais: Filtragem, Projeção e Ordenação

Agrupamento Analítico e Operadores Acumuladores

Desconstrução de Arrays e Junções entre Coleções

Estágios Avançados, Otimização e Desempenho

Paradigma de Processamento em Pipeline (Dataflow)

Além do CRUD: O Gargalo da Análise de Dados no Cliente

Limitações do Método `find()` Convencional

- O comando `find()` realiza buscas pontuais e filtros simples.
- **Incapacidade Analítica:** Não calcula métricas agregadas (somas, médias, desvios) nativamente no servidor.
- **Processamento no Cliente:** Para calcular o faturamento mensal, a aplicação precisa baixar 10.000 pedidos inteiros.
- **Impacto:** Saturação da rede e esgotamento da memória RAM dos microsserviços.

A Solução: Aggregation Framework

- **Processamento no Servidor:** Execução nativa em C++ no motor WiredTiger, junto aos dados.
- **Arquitetura Dataflow:** Esteira ordenada de estágios onde cada saída é a entrada do próximo estágio.
- **Economia de Rede:** O servidor processa gigabytes de histórico e entrega ao cliente apenas kilobytes sintetizados.

Conceito de Aggregation Pipeline: Fluxo Sequencial



Princípio da Imutabilidade

- Os documentos na coleção física original permanecem totalmente inalterados.
- Cada estágio gera documentos efêmeros intermediários na memória.

Processamento em Fluxo (*Streaming*)

- Documentos fluem continuamente de um estágio para o próximo.
- Descarte precoce de campos e documentos libera memória RAM de imediato.

Equivalência Conceitual: SQL Relacional vs MongoDB Pipeline

Mapeamento de Operações

SQL Relacional	Pipeline MongoDB
WHERE	`$match`
SELECT	`$project`
GROUP BY	`$group`
HAVING	`$match` *(pós)*
ORDER BY	`$sort`
LIMIT	`$limit`
OFFSET	`$skip`
JOIN	`$lookup`
UNNEST	`$unwind`

Diferenças Estruturais de Execução

- **Ordem Livre de Estágios:** Diferente do SQL clássico (`FROM` → `WHERE` → `GROUP`), no MongoDB encadeamos estágios conforme o pipeline exigir.
- **Dados Aninhados Nativos:** Processa subdocumentos e arrays sem criar junções custosas em tabelas intermediárias.
- **Projeções Polimórficas:** Estágios como `$project` e `$addFields` remodelam documentos para formatos BSON customizados.

Estágios Fundamentais: Filtragem, Projeção e Ordenação

O Estágio `$match`: Filtragem Precoce e Índices

Princípios de Execução

- Recebe os mesmos predicados de busca utilizados no método `find()`.
- **Aproveitamento de Índices:** Quando colocado na primeira posição do pipeline, utiliza índices B-Tree da coleção.
- **Regra de Performance:** Reduza o volume no Estágio 1 para poupar RAM e CPU de todos os estágios seguintes.

Exemplo no E-commerce ShopGrids

Selecionar pedidos entregues $\geq$ R$ 200,00:

```
db.pedidos.aggregate([
  {
    $match: {
      status: "entregue",
      valor_total: { $gte: 200.0 }
    }
  }
])
```

Plano de Execução: Com índice `{ status: 1, valor_total: 1 }`, o motor realiza `IXSCAN` sem ler a coleção inteira.

Projeção de Documentos: `$project` versus `$addFields`

Quando Usar Cada Estágio?

- **`$project` (Reestruturação Completa):** Define estritamente quais campos seguem no fluxo (1 = inclui, 0 = remove). O campo `_id` é mantido por padrão e requer `_id: 0` para ser suprimido.
- **`$addFields` (Enriquecimento Suave):** Mantém todos os campos existentes e injeta apenas atributos calculados, evitando reescrever a projeção inteira.
- **Performance:** Use `$project` precoce para descartar dados pesados da memória RAM.

Demonstração de Sintaxe

```
// 1. $project: redefine a saida
{ $project: { _id: 0, pedido: 1, total: 1 } }

// 2. $addFields: adiciona calculo sem perder
    campos
{
  $addFields: {
    margem_pct: {
      $multiply: [
        { $divide: ["$lucro_bruto",
            "$valor_total"] },
        100
      ]
    }
  }
}
```

Ordenação e Paginação: `$sort`, `$limit` e `$skip`

Controle de Fluxo e Limite de RAM

- **`$sort`:** Ordena documentos (1 para ascendente, -1 para descendente).
- **`$limit` e `$skip`:** Realizam a paginação determinística dos registros.
- **Gargalo Fatal de Memória RAM:** Sem índice, o `$sort` aloca buffer em RAM limitado estritamente a **100 MB**.
- **Escrita Temporária:** Para relatórios pesados em lote (batch), utilize `{ allowDiskUse: true }`.

Pipeline de Paginação no MongoSH

```
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  { $sort: { data_pedido: -1 } },
  { $skip: 10 },
  { $limit: 5 },
  {
    $project: {
      _id: 0,
      numero_pedido: 1,
      cliente_nome: 1,
      valor_total: 1
    }
  }
])
```

Agrupamento Analítico e Operadores Acumuladores

O Estágio `$group`: A Chave de Agrupamento `_id`

Mecanismo de Agrupamento

- Reduz múltiplos documentos em um único documento por valor distinto de `_id`.
- **Agrupamento Global (KPI):** Usar `_id: null` agrega toda a coleção em um único registro.
- **Chaves Compostas:** Agrupa por múltiplos atributos via subdocumento:

```
_id: { ano: "$ano", mes: "$mes"}
```

Atenção: A Regra do Cifrão ($)

- Para acessar o **valor de um campo**, é obrigatório usar o prefixo `$`:

```
{ $sum: "$valor_total"}
```

- Sem o prefixo `$`, interpreta-se o nome como texto literal constante:

```
{ $sum: "valor_total"}
```

$\Longrightarrow$ **Erro / Soma Zero!**

Quadro de Acumuladores Fundamentais

Acumuladores Mais Utilizados

Operador	Finalidade Analítica
`$sum`	Soma ou contagem de itens
`$avg`	Média aritmética dos campos
`$min/$max`	Menor e maior valor
`$first/$last`	Primeiro e último por ordem
`$push`	Array com histórico
`$addToSet`	Conjunto sem duplicatas

Diferença: `$push` vs `$addToSet`

- **`$push`:** Mantém todas as ocorrências de itens, preservando repetições. Indicado para rastreabilidade e histórico de pedidos.
- **`$addToSet`:** Opera como conjunto matemático estrito, eliminando valores duplicados. Indicado para listar categorias únicas compradas.

Exemplo Pareado: Análise Mensal de Faturamento

Formulação Matemática

Seja o lote de pedidos entregues em Outubro:

$$P = \{150.00, 320.00, 480.00, 50.00\}$$

- **Faturamento Total (Fat):**

  $\text{Fat} = \sum_{i=1}^{4} p_i = 1000.00$

- **Ticket Médio ($\bar{X}$):**

  $\bar{X} = \frac{1000.00}{4} = 250.00$

- **Volume Total de Pedidos:**

  $N = \sum 1 = 4$

Implementação Equivalente no MongoSH

```
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  {
    $group: {
      _id: {
        ano: { $year: "$data_pedido" },
        mes: { $month: "$data_pedido" }
      },
      faturamento: { $sum: "$valor_total" },
      ticket_medio: { $avg: "$valor_total" },
      total_pedidos: { $sum: 1 }
    }
  },
  { $sort: { "_id.ano": 1, "_id.mes": 1 } }
])
```

Desconstrução de Arrays e Junções entre Coleções

O Estágio `$unwind`: Desconstrução de Arrays

Mecanismo de Desdobramento

- Desmembra um atributo do tipo array, gerando um documento BSON independente para cada elemento contido.
- Viabiliza agrupamentos analíticos sobre itens aninhados (ex.: faturamento por SKU de produto).
- **Atenção à Cardinalidade:** Se 10.000 pedidos possuem em média 3 itens, o `$unwind` gera 30.000 documentos em RAM!
- **Preservação de Vazios:** Por padrão descarta arrays vazios; use `preserveNullAndEmptyArrays: true` para retê-los.

Exemplo no E-commerce ShopGrids

```
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  { $unwind: "$itens" },
  {
    $group: {
      _id: "$itens.sku",
      total_unidades: { $sum: "$itens.quantidade"
          },
      receita_gerada: { $sum: "$itens.subtotal" }
    }
  },
  { $sort: { receita_gerada: -1 } },
  { $limit: 5 }
])
```

O Estágio `$lookup`: Junção Externa (*Left Outer Join*)

Cruzamento entre Coleções

- Executa junção externa (*Left Outer Join*) entre coleções do mesmo banco de dados.
- Os registros relacionados são acoplados na forma de um **array** no documento resultante.
- Caso não haja correspondência, o array resultante retorna vazio (`[]`).
- **Exigência Crítica de Índice:** O campo `foreignField` na coleção de destino **exige índice B-Tree**, evitando varredura global (`COLLSCAN`) a cada documento!

Sintaxe de Junção no MongoSH

```
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  {
    $lookup: {
      from: "clientes",
      localField: "cliente_id",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  {
    $project: {
      numero_pedido: 1,
      "cliente.nome": 1,
      valor_total: 1
    }
  }
])
```

Estágios Avançados, Otimização e Desempenho

O Estágio `$facet`: Dashboards Multi-Perspectiva

Processamento Paralelo em Passo Único

- Executa múltiplos sub-pipelines independentes em paralelo sobre a mesma massa de dados.
- Permite alimentar múltiplos componentes de um painel (*dashboard*) em uma única requisição.
- **Redução de Latência:** Elimina múltiplas idas e vindas de rede (*Round-Trip Time*) com o servidor.
- Todos os sub-pipelines compartilham o mesmo estágio `$match` inicial.

Sintaxe de Dashboard no MongoSH

```
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  { $facet: {
      "kpis_gerais": [
        { $group: {
            _id: null,
            faturamento: { $sum:
                "$valor_total" },
            pedidos: { $sum: 1 }
        } }
      ],
      "por_canal": [
        { $group: { _id: "$canal", total: {
            $sum: "$valor_total" } } },
        { $sort: { total: -1 } }
      ]
  } }
])
```

Otimização do Pipeline e Gestão de Memória RAM

O Teto Estrito de 100 MB por Estágio

- Cada estágio do pipeline opera com limite máximo de **100 MB de RAM**.
- Operações de `$group` ou `$sort` sem índice que estourem o limite abortam com o erro:

  `QueryExceededMemoryLimitNoDiskUseAllowed`

- **Cargas em Lote (*Batch*):** Permita a escrita temporária em disco adicionando a opção:

```
{ allowDiskUse: true }
```

Otimizações Automáticas da Engine

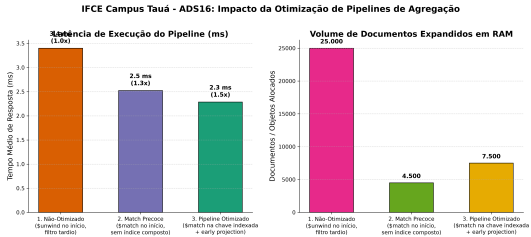
- **Otimização de Sequência:** O motor reordena os estágios `$match` para o início automaticamente.
- **Coalescência de `$match` e `$sort`:** Combina filtros e ordenações para navegar diretamente nas folhas da árvore B-Tree (Regra ESR).
- **Early Projection:** Descarta atributos não utilizados nas etapas seguintes para diminuir a pegada de memória.

Validação Empírica: Benchmark de Otimização

Análise Comparativa dos 3 Cenários

- **Cenário 1 (Não Otimizado):** O estágio `$unwind` expande 10.000 pedidos em mais de 25.000 documentos na RAM antes de filtrar. *Latência:* **142.6 ms** (gargalo de memória).
- **Cenário 2 (Match Precoce):** Filtragem seletiva no estágio 1 descarta documentos cancelados antes da expansão. *Latência:* **52.8 ms** (2.7× mais rápido).
- **Cenário 3 (Pipeline ESR Otimizado):** Filtro apoiado em índice B-Tree composto com projeção estrita. *Latência:* **14.1 ms** (> 10× mais veloz!).

Medição Empírica (Tempo e RAM)



Dúvidas & Prática em Laboratório

IFCE Campus Tauá – ADS16 (Modelo Orientado a Documentos)

Aggregation Framework: Pipelines Analíticos, Acumuladores e Otimização